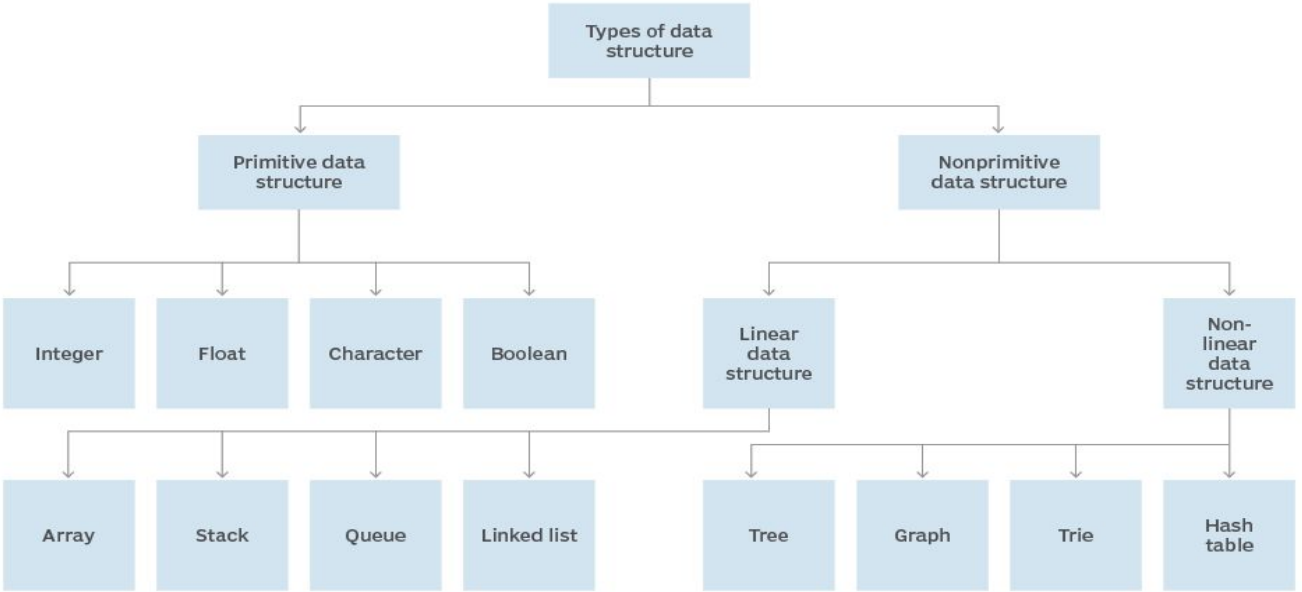
Data Structure

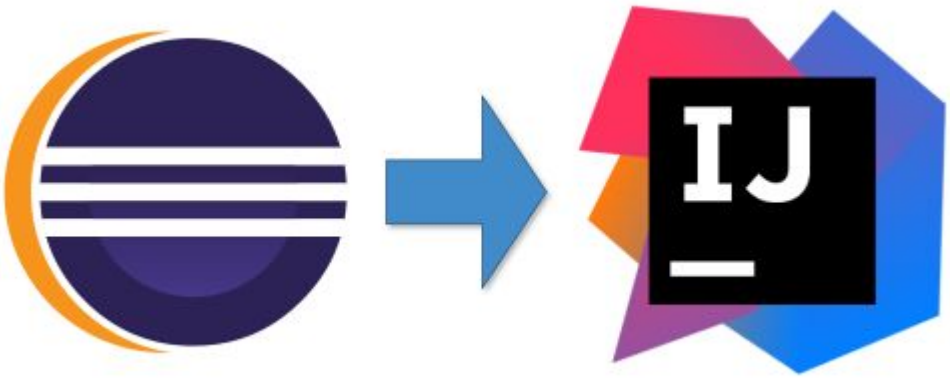
Padmaraj Nidagundi

Last class.....

Data structure hierarchy

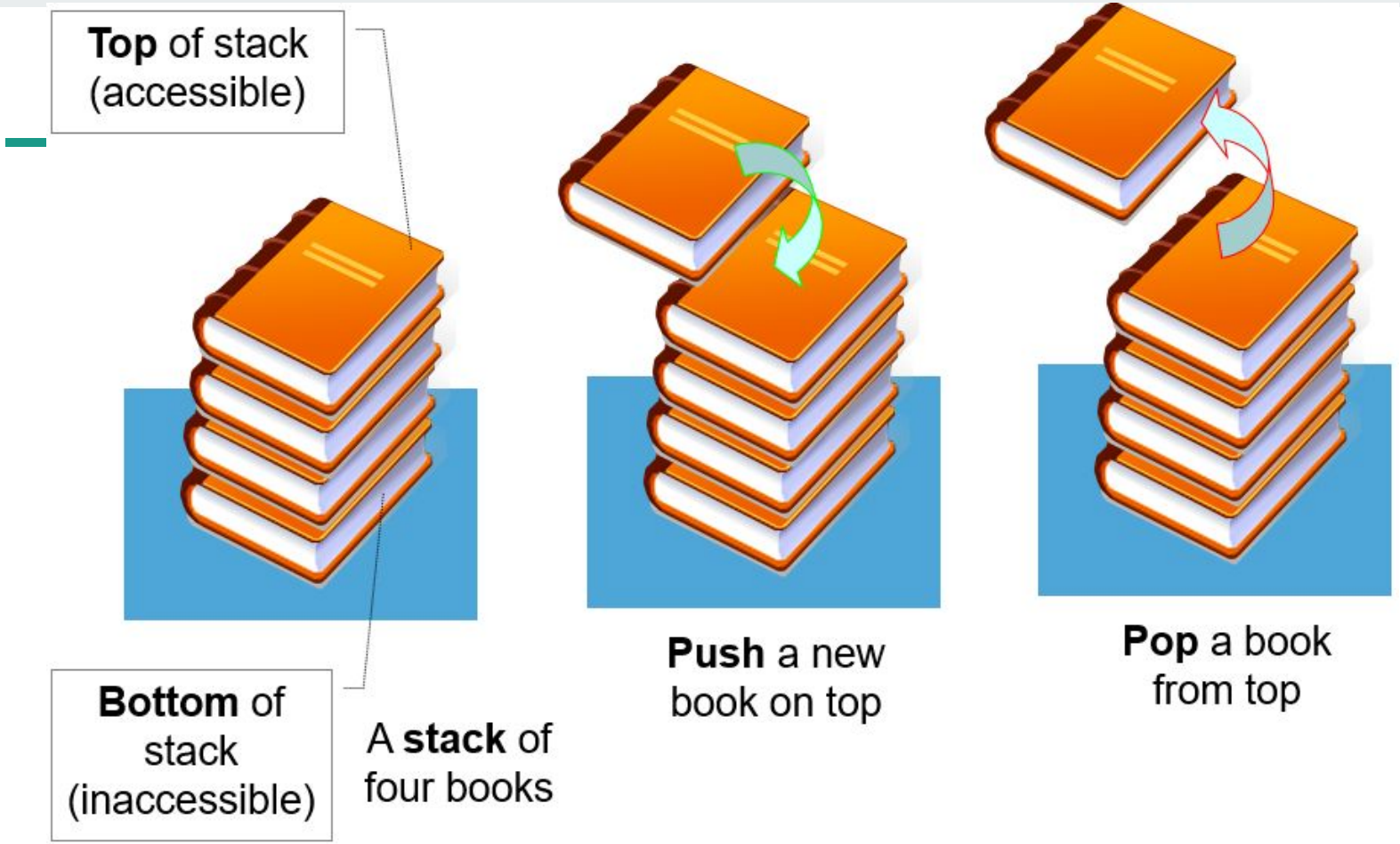


Student must need to use IDE



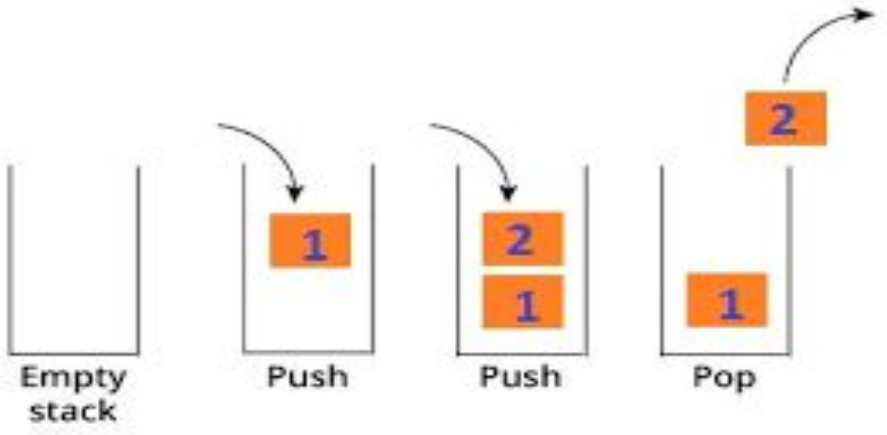
Light weight Online Java Compiler - https://www.codiva.io/
https://paiza.io/en/projects/new?language=java

STACK - LIFO


Top of stack (accessible)
Bottom of stack (inaccessible)
A **stack** of four books
Push a new book on top
Pop a book from top

STACK

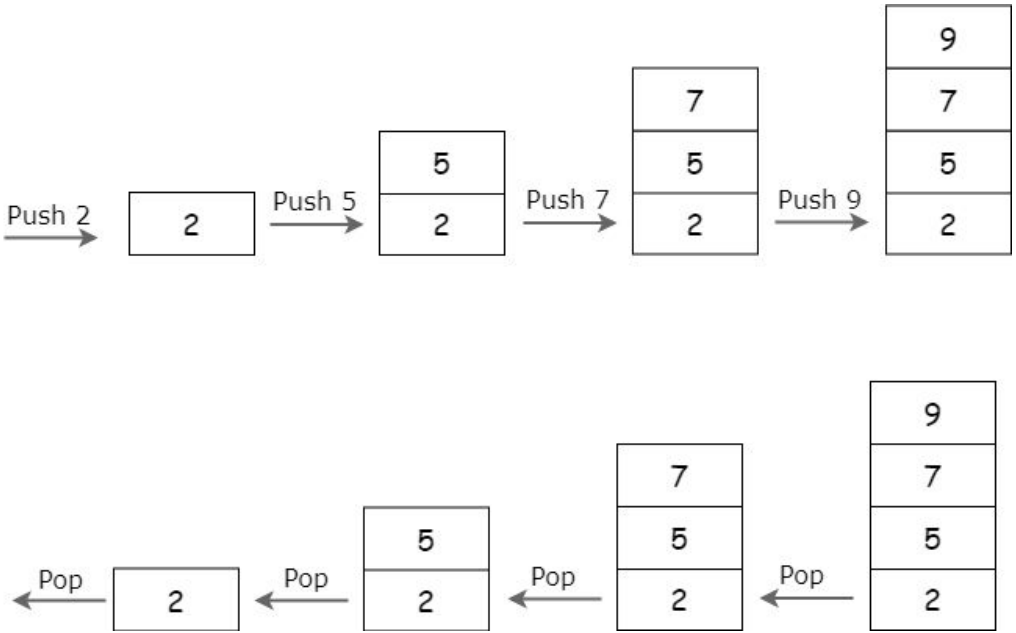
Stack. This collection is implements **LIFO**: the last in is the first out. We push elements onto the top of a stack and then pop them for retrieval.



LIFO

Stack: Array Implementation Visually - Example

https://www.cs.usfca.edu/~galles/visualization/StackArray.html



Stack Operations

Stack operations may involve initializing the stack, using it and then de-initializing it. Apart from these basic stuffs, a stack is used for the following two primary operations −

- **push()** − Pushing (storing) an element on the stack.
- **pop()** − Removing (accessing) an element from the stack.

When data is PUSHed onto stack.

To use a stack efficiently, we need to check the status of stack as well. For the same purpose, the following functionality is added to stacks −

- **peek()** − get the top data element of the stack, without removing it.
- **isFull()** − check if stack is full.
- **isEmpty()** − check if stack is empty.

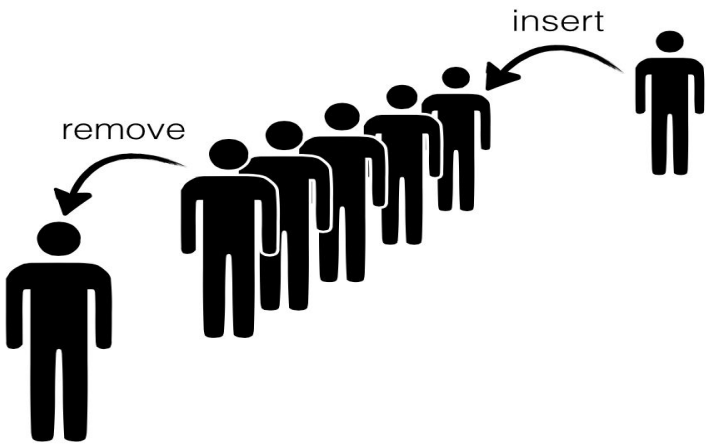
At all times, we maintain a pointer to the last PUSHed data on the stack. As this pointer always represents the top of the stack, hence named **top**. The **top** pointer provides top value of the stack without actually removing it.

Applications of Stack

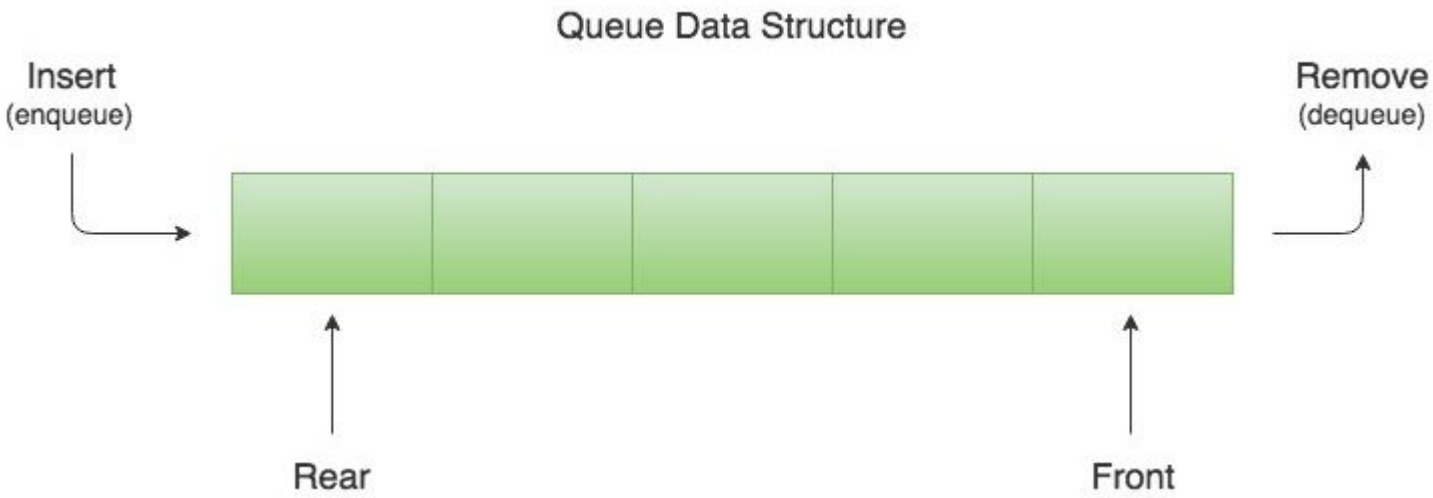
- Expression Evaluation. **Stack** is used to evaluate prefix, postfix and infix expressions.
- Expression Conversion. An expression can be represented in prefix, postfix or infix notation. ...
- Syntax Parsing. ...
- Backtracking. ...
- Parenthesis Checking. ...
- Function Call.

Queue - FIFO

Queue Representation



A Queue is a First In First Out (FIFO) data structure. It models a queue in real-life. Yes, the one that you might have seen in front of a movie theater, a shopping mall, a metro, or a bus.



Operations on Queue

add()- This method is used to add elements at the tail of queue. More specifically, at the last of linked-list if it is used, or according to the priority in case of priority queue implementation.

peek()- This method is used to view the head of queue without removing it. It returns Null if the queue is empty.

element()- This method is similar to peek(). It throws *NoSuchElementException* when the queue is empty.

remove()- This method removes and returns the head of the queue. It throws *NoSuchElementException* when the queue is empty.

poll()- This method removes and returns the head of the queue. It returns null if the queue is empty.

size()- This method return the no. of elements in the queue.

OPERATION	THROWS EXCEPTION	RETURN VALLUES
Insert	add(element)	offer(element)
Remove	remove()	poll()
Examine	element()	peek()

Applications of Queue

- Serving requests on a single shared resource, like a printer, CPU task scheduling etc.
- In real life scenario, Call Center phone systems uses **Queues** to hold people calling them in an order, until a service representative is free.
- Handling of interrupts in real-time systems.

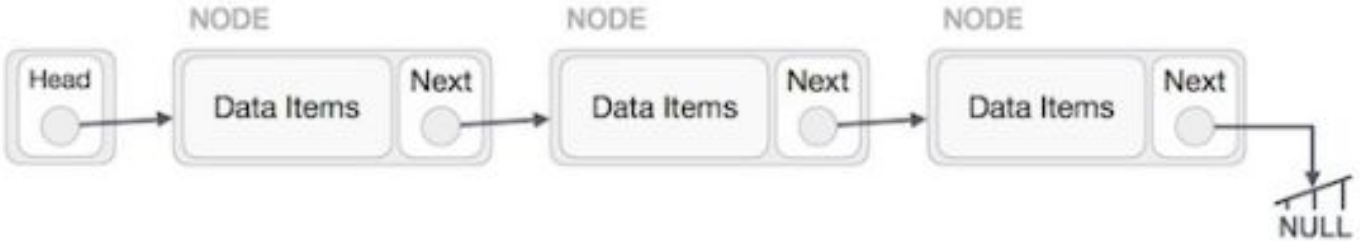
Today's Topic: Linked List

Linked List

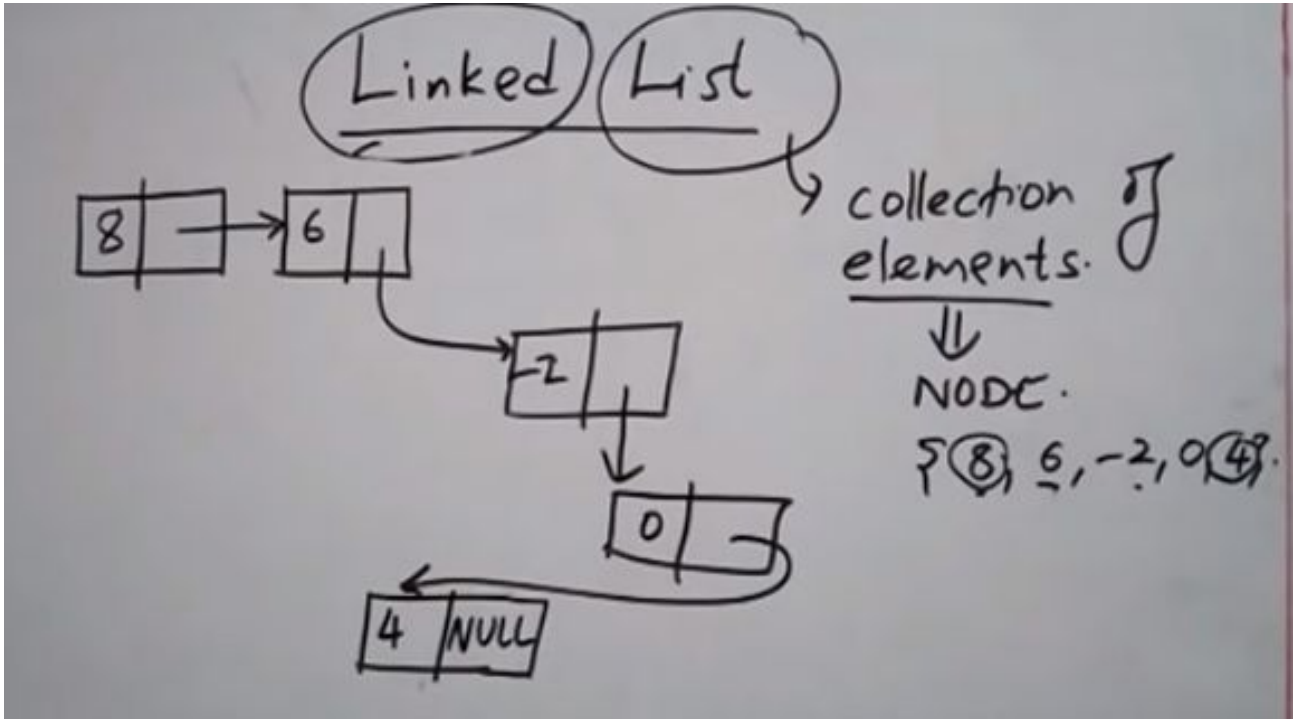
A linked list is a linear data structure where each element is a separate object.

Linked List is a sequence of links which contains items. Each link contains a connection to another link. Linked list is the second most-used data structure after array. Following are the important terms to understand the concept of Linked List.

- **Link** – Each link of a linked list can store a data called an element.
- **Next** – Each link of a linked list contains a link to the next link called Next.
- **LinkedList** – A Linked List contains the connection link to the first link called First.



- Linked List contains a link element called first.
- Each link carries a data field(s) and a link field called next.
- Each link is linked with its next link using its next link.
- Last link carries a link as null to mark the end of the list.


Linked List
collection of elements.
NODE.
{8, 6, -2, 0, 4}.
8
6
-2
0
4
NULL

Types of Linked List

Following are the various types of linked list.

- **Simple Linked List** – Item navigation is forward only.
- **Doubly Linked List** – Items can be navigated forward and backward.
- **Circular Linked List** – Last item contains link of the first element as next and the first element has a link to the last element as previous.

Basic Operations

Following are the basic operations supported by a list.

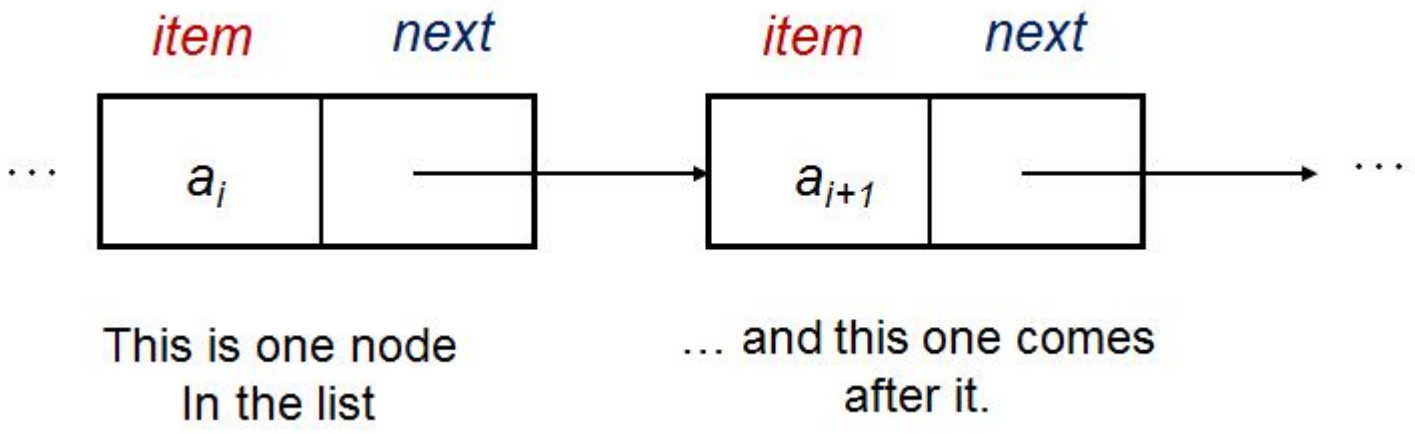
- **Insertion** – Adds an element at the beginning of the list.
- **Deletion** – Deletes an element at the beginning of the list.
- **Display** – Displays the complete list.
- **Search** – Searches an element using the given key.
- **Delete** – Deletes an element using the given key.

Applications of linked list data structure

- Implementation of stacks and queues.
- Implementation of graphs : Adjacency **list** representation of graphs is most popular which is uses **linked list** to store adjacent vertices.
- Dynamic memory allocation : We use **linked list** of free blocks.
- Maintaining directory of names.
- Performing arithmetic operations on long integers.

Visualised linked list

https://visualgo.net/en/list



ARRAY	LINKED LIST
Size of an array is fixed	Size of a list is not fixed
Memory is allocated from stack	Memory is allocated from heap
It is necessary to specify the number of elements during declaration (i.e., during compile time).	It is not necessary to specify the number of elements during declaration (i.e., memory is allocated during run time).
It occupies less memory than a linked list for the same number of elements.	It occupies more memory.
Inserting new elements at the front is potentially expensive because existing elements need to be shifted over to make room.	Inserting a new element at any position can be carried out easily.
Deleting an element from an array is not possible.	Deleting an element is possible.

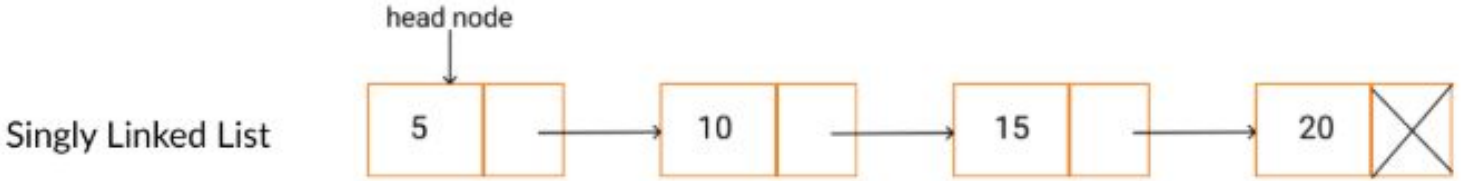
	Array	Linked List
Strength	• Random Access (Fast Search Time) • Less memory needed per element • Better cache locality	• Fast Insertion/Deletion Time • Dynamic Size • Efficient memory allocation/utilization
Weakness	• Slow Insertion/Deletion Time • Fixed Size • Inefficient memory allocation/utilization	• Slow Search Time • More memory needed per node as additional storage required for pointers

```java
import java.util.LinkedList;
import java.util.List;

public class LinkedListDemo
{
  public static void main(String[] args)
  {
        List names = new LinkedList();
        names.add("Rams");
        names.add("Posa");
        names.add("Chinni");
        names.add(2011);

        System.out.println("LinkedList content: " + names);
        System.out.println("LinkedList size: " + names.size());
  }
}
```

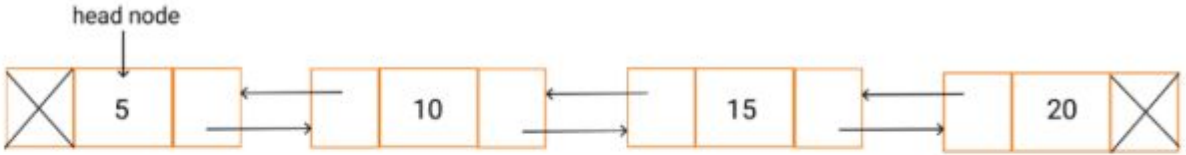
```
LinkedList content: [Rams, Posa, Chinni, 2011]
LinkedList size: 4
```

Types of Linked List

The single linked list contains only one link for each node which points to the next link and therefore can navigate only forward direction .

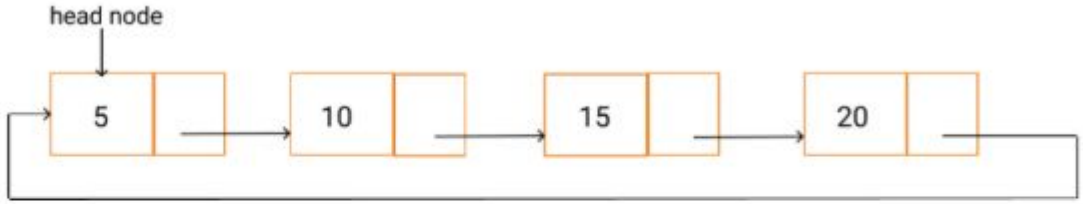


Doubly Linked List



The double linked list contains two link fields for each node , The first link points to previous node and the second link points to the next link . And therefore in case of double linked list , it is possible navigation in both backward and forward direction .

Circular Linked List



In circular linked list , the last node contains link next , which points to the value field of the first node and the first node link field has a link as previous , which points to the last node value field .

Home Task: Write 5 LinkedList example program

Examples here - https://www.journaldev.com/13386/java-linkedlist-linkedlist-java